LoRaWAN网络

1. 系统组成

1. **端设备：**终端设备通过无线网关无线连接到 LoRaWAN 网络。
2. **网关：**网关将所有接收到的 LoRaWAN无线电数据包转发到网络服务器
3. **网络服务器：**网络服务器是星型拓扑的中心，它运行在管理整个系统的服务器上
4. **应用服务器：**应用服务器处理相关终端设备的所有应用层有效负载，并向最终用户提供应用级服务

1.1.1 端设备

端设备是LoRaWAN网络中的终端节点，负责生成和接收数据，它们通常由电池供电，可以工作在不同的频段，

a. 功能:与网关设备进行数据交互，将传感器数据发送到网关和接收网关的数据
b. 特性：以下特性对于每台设备都是唯一的，一般通过烧录固件时确定，网关通过以下特性连接与区分终端设备,可以认为是端设备身份信息
    i. **LORAWAN_DEVICE_EUI**（设备唯一标识符）：
        1. 每个LoRaWAN设备都有一个全球唯一的64位标识符，称为设备EUI（Device EUI）。
        2. 设备EUI在设备出厂时被烧录到设备中，并且是不可更改的。
        3. 它用于在网络中唯一标识该设备。
    ii. **LORAWAN_JOIN_EUI**（应用服务提供商标识符）：
        1. 应用服务提供商（如TTN、ChirpStack等）会有一个唯一的8位标识符，称为应用EUI（Join EUI）。
        2. 设备需要使用这个应用EUI来加入到对应的LoRaWAN网络中。
        3. 这个参数通常是由网络运营商或应用服务提供商提供的。
    iii. **LORAWAN_APP_KEY**（应用密钥）：
        1. 应用密钥是用于LoRaWAN设备与网络服务器之间加密通信的密钥。
        2. 有两个密钥：一个应用密钥（App Key）和一个应用密钥（App Key）。

3. 这些密钥用于数据的加密和解密，确保数据传输的安全性。
4. 它们应该是安全生成的随机数，并且对网络中的每个设备都是唯一的。

iv. **LORAWAN_NWK_KEY**（网络会话密钥）：

1. 网络会话密钥（Nwk Key）也是用于LoRaWAN设备之间加密通信的密钥。
2. 它与应用密钥类似，用于数据的加密和解密，但它是用于网络层的安全。
3. 类似于应用密钥，网络会话密钥也应该是安全生成的随机数，并且对于每个设备应该是唯一的。

```
比如:
MAC version: 1.0.3          #支持LoRa协议版本
Regional Parameters: B      #区域参数
Region: CN470               #区域，"CN470"是指中国的一个特定频段
Class: Class A              #设备类型
DEUI: 00,80,E1,15,00,0A,DA,51
DevAddr: 01,FB,26,33
JOIN_EUI/APP_EUI: 02,02,02,02,02,02,02,02
AppKey/NwkKey/NwkSKey/AppSKey:
2B,7E,15,16,28,AE,D2,A6,AB,F7,15,88,09,CF,4F,30
```

1.1.2 网关

网关是LoRaWAN网络中的一个设备，它能够将LoRaWAN终端设备发送的数据转发到网络服务器，也可接受网络服务器数据下发到LoRaWAN终端设备

1.1.3 网络服务器

网络服务器(Network Server)在LoRaWAN（Long Range Wide Area Network）网络架构中扮演着非常重要的角色。它是负责处理LoRaWAN网络中设备的 Join、数据传输、以及设备的管理等任务的服务器。

功能

i. **设备管理**

1. **设备注册**：Network Server负责处理设备的注册请求，确保设备能够合法地加入LoRaWAN网络。
2. **设备认证**：通过设备的唯一标识符（DevEUI）和认证码（AppKey）来验证设备的合法性。
3. **设备会话管理**：维护设备的会话信息，例如会话密钥（Session Keys），以确保设备间的安全通信

ii. **数据传输**

1. **上行数据**：Network Server接收来自设备的上行数据，并将其转发到应用服务器或其他设备。
2. **下行数据**：Network Server将应用服务器发送的下行数据下发给特定的设备。
3. **数据多路复用**：Network Server负责处理多设备同时发送数据到同一网关的情况，实现数据的多路复用

iii. **网关管理**

1. **网关注册**：Network Server需要管理网关的注册信息，以便能够将数据包正确地路由到正确的网关。
2. **网关数据转发**：网关将接收的数据包转发到Network Server，由Network Server进行进一步的处理。

iv. **网络控制**

1. **数据速率和频率规划**：Network Server根据网络的负载情况，为设备选择合适的数据速率和频率。
2. **ADR（自适应数据速率）**：根据设备的接收信号质量，Network Server调整数据速率以优化网络性能。

v. **安全性**

1. **数据加密**：Network Server负责对设备间的数据进行加密，确保数据的安全性。
2. **设备认证**：Network Server通过验证设备的身份来确保只有合法的设备能够连接到网络。

1.1.4 应用服务器

应用服务器是一款开源 LoRaWAN (应用服务器)，是开源 LoRaWAN 网络服务器堆栈的一部分。它负责 LoRaWAN 基础设施的设备“清单”部分、加入请求的处理以及应用程序有效负载的处理和加密。它提供了一个Web界面 ，可以管理用户、组织、应用程序和设备。为了与外部服务集成，它提供了gRPC。设备数据可以通过 MQTT、HTTP转发出去。

1. **有效载荷加密/解密：** 应用服务器负责处理应用程序有效负载的加密和解密。它还保存每个设备的应用程序密钥，并在 OTAA 激活的情况下处理加入接受。这意味着有效负载将以解密形式发送到集成，但在将有效负载发送到网络服务器之前，网络服务器无法访问这些有效负载。
2. **网页界面**: 应用服务器提供 Web 界面。此 Web 界面可用于管理用户、组织、应用程序和设备。
3. **用户授权**: 可以授予用户全局管理员权限，使他们成为组织的管理员或为他们分配组织内的只读权限。这使得可以在多租户环境中运行应用服务器，其中每个组织或团队只能访问自己的应用程序和设备。
4. **API:** 为了与外部服务集成，一般应用服务器提供了gRPCAPI，它公开与 Web 界面相同的功能。

1.1.5 cloud

接收应用服务器转发的设备数据,通过应用服务器提供API与其交流

2. 网络/应用服务器

ChirpStack是一个开源的LoRaWAN网络服务器（Network Server）可用于配置 LoRaWAN 网络

参考：https://www.chirpstack.io/project/

2.1 核心组件

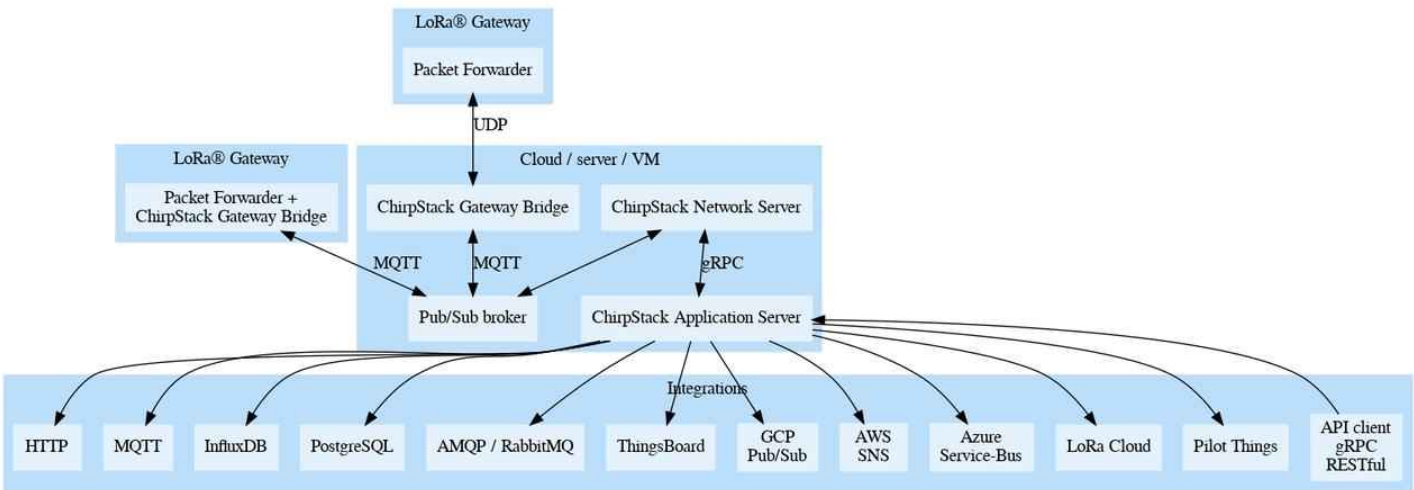
- ChirpStack Gateway Bridge： 网关桥接器是一种将 LoRa 数据包转发器协议转换为 ChirpStack Network Server（JSON 和 Protobuf）的服务
- ChirpStack Network Server ：网络服务器
- ChirpStack Application Server：应用服务器

备注

```
ChirpStackv4已经将chirpstack-network-server和chirpstack-application-server合并起来变成chirpstack
```

2.2 ChirpStack LoRaWAN 网络服务器组件的连接方式：

https://www.chirpstack.io/project/architecture/



ChirpStack Application Server

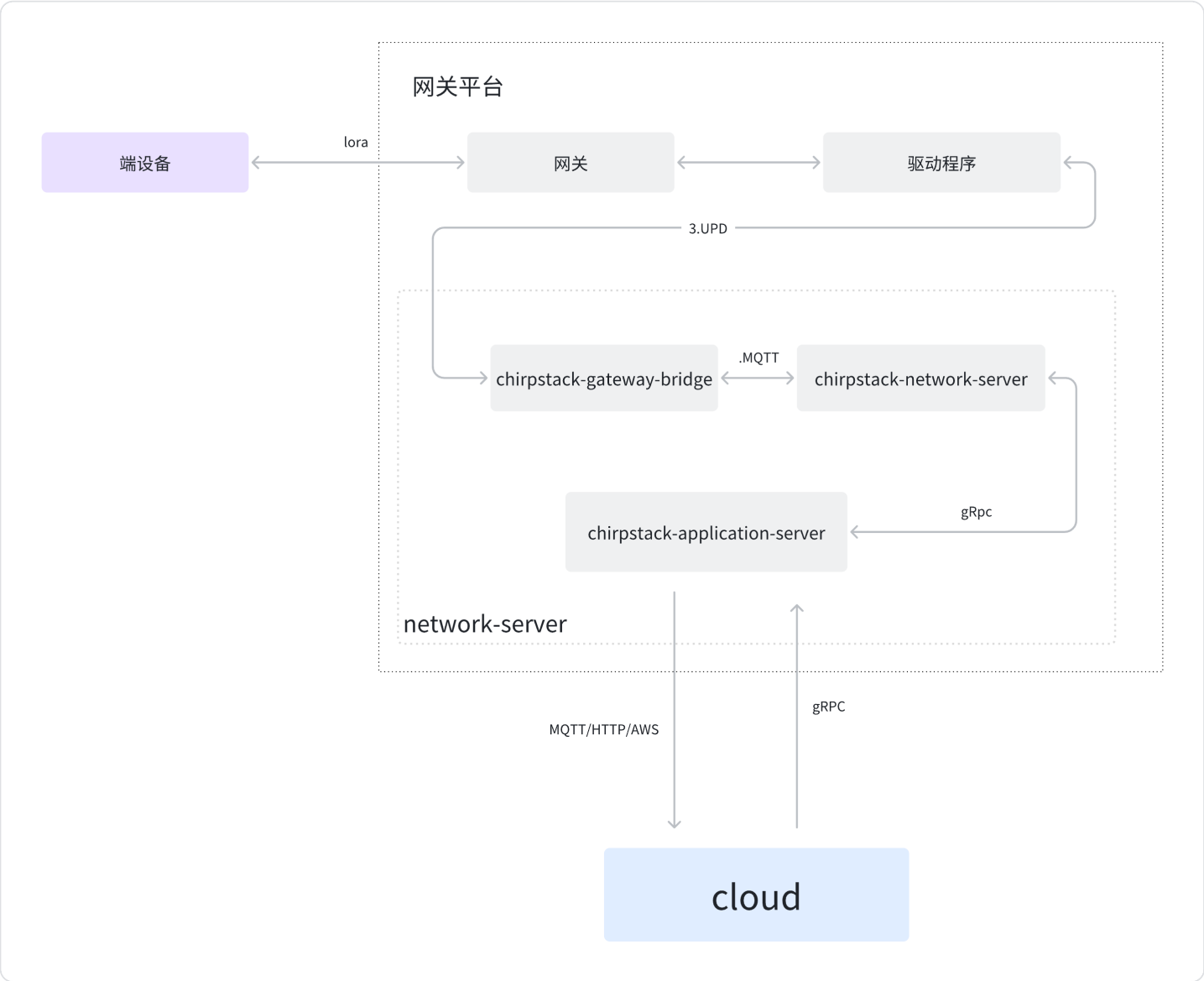
ChirpStack Application Server（也称为ChirpStack AS或AS服务器）是一个组件，它位于LoRaWAN网络的核心，负责处理来自设备的上行数据，以及发送下行数据给设备。它还负责设备

的管理和数据的转发给集成系统，如用户自定义的Webhooks或MQTT broker。它提供 Web 界面和 API，用于管理用户、组织、应用程序、网关和设备。接收到的上行数据被转发到一个或多个配置的集成

3. 端设备数据到云流程

3.1 网络服务器与网关集成部署

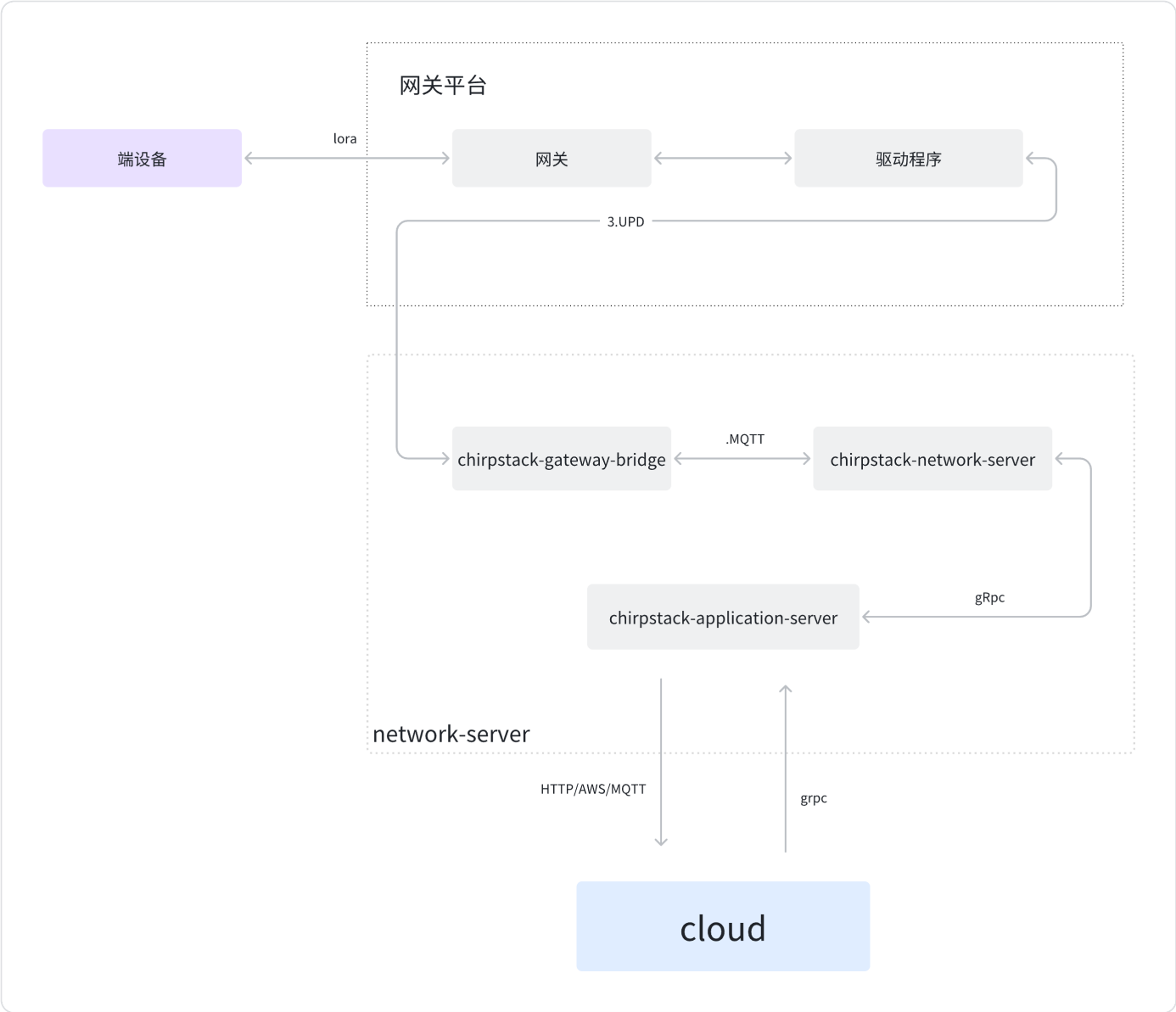
网络服务器，网关部署在同一设备上，与云端通讯通过MQTT/gRPC



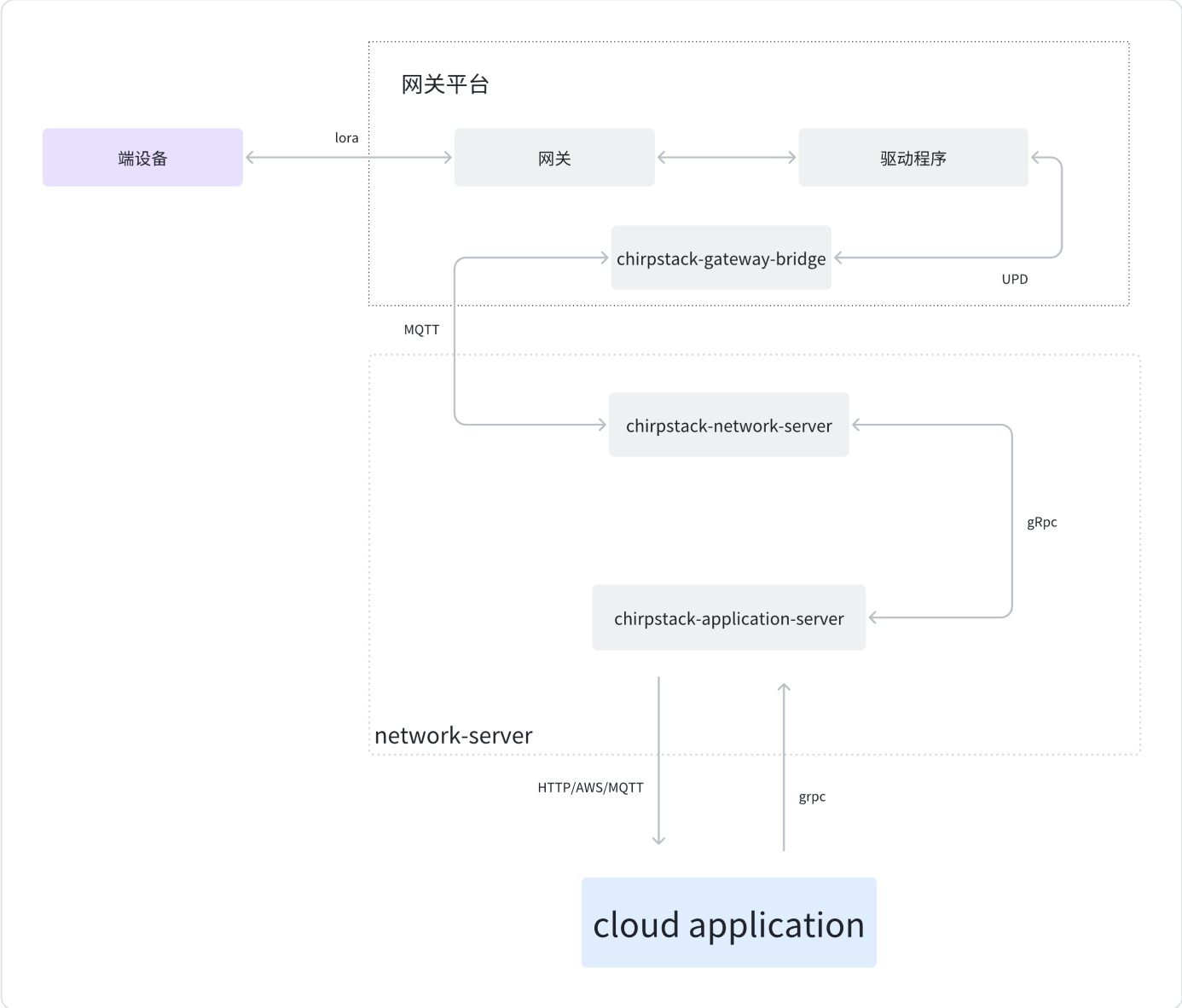
3.2 网络服务器与网关分离部署

网络服务器与网关分离部署，通过MQTT进行通讯，网络服务器与云端通讯通过MQTT/gRPC

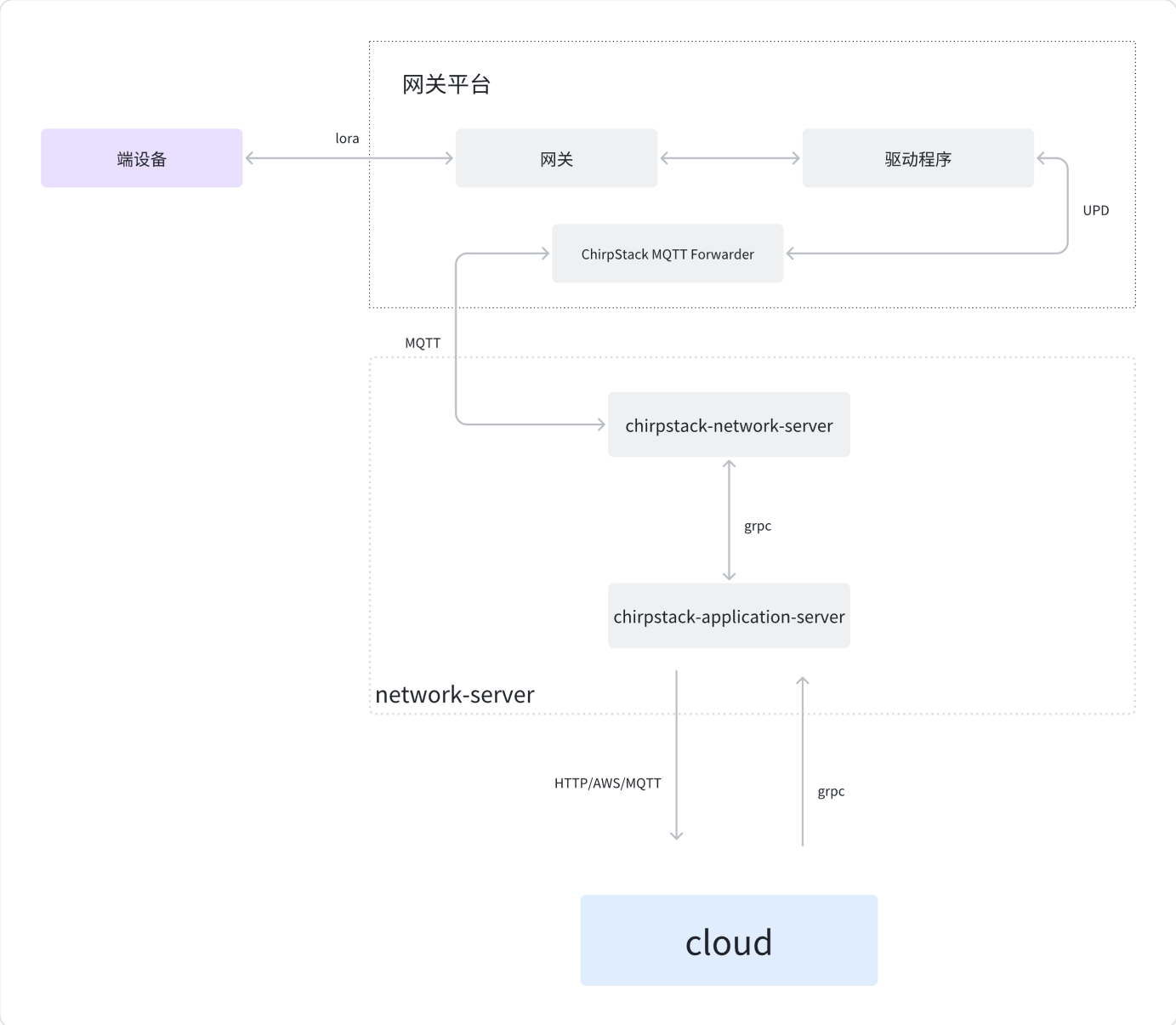
a. UDP Packet Forwarder



b. UDP Packet Forwarder + chirpstack-gateway-bridge



c. UDP Packet Forwarder + ChirpStack MQTT Forwarder



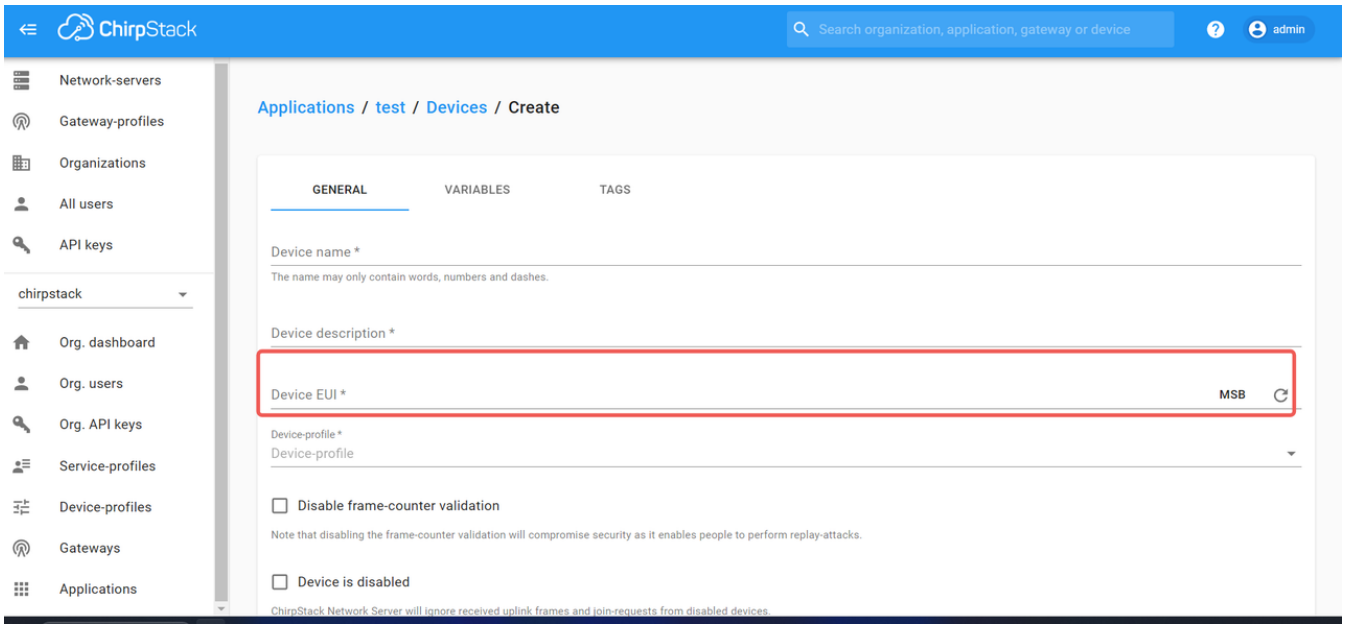
4. 如何确保数据安全

4.1.1 端设备如何安全连接到网关

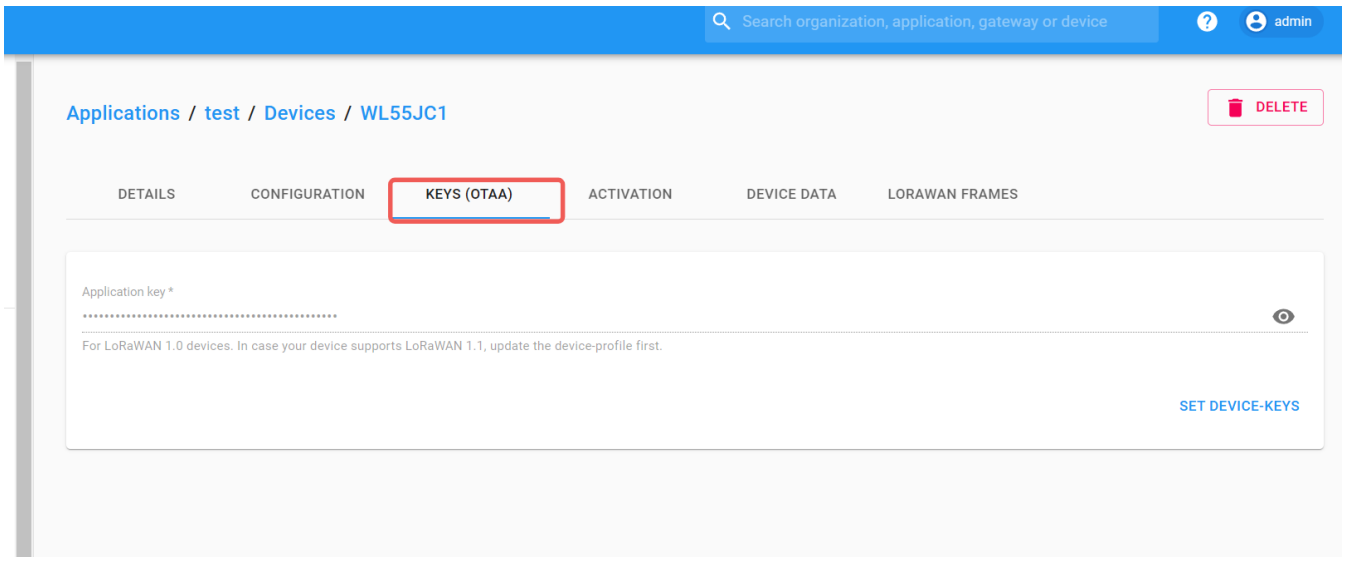
1. 端设备出厂烧录身份信息(LoRaWAN网络)，
   a. 如何配置烧录身份信息

```
https://wiki.st.com/stm32mpu-ecosystem-v3/wiki/How_to_integrate_LoRaWAN_gateway#How_to_configure_Nucleo-WL55JC_as_LoRa_end_node_device
```

2. 网络服务器获取端设备身份信息并配置到网关
   a. ChirpStack如何配置
      i. 配置eui



ii. keys



3. 网关通过端设备身份信息与端设备配对连接
4. 端设备上传下载数据

4.1.2 设备数据如何安全流转至云端

1. 端设备<->网关：连接阶段密钥已经交换，数据传输就已经是密文了
2. 网关<-> 网络服务器:
   a. UDP：需要额外提供算法对源数据加密解密
   b. MQTT:本身可通过配置(用户密码证书等)确保安全性
3. 网络服务器-->云端
   a. MQTT:本身可通过配置(用户密码证书等)确保安全性
4. 云端-->网络服务器
   a. ChirpStack通过api gRPC可以配置token

5. 设备如何区分

整个系统就绪后云端收到此类型消息,通过以下字段则可以知道此设备是哪个设备，部署在哪个网关上，

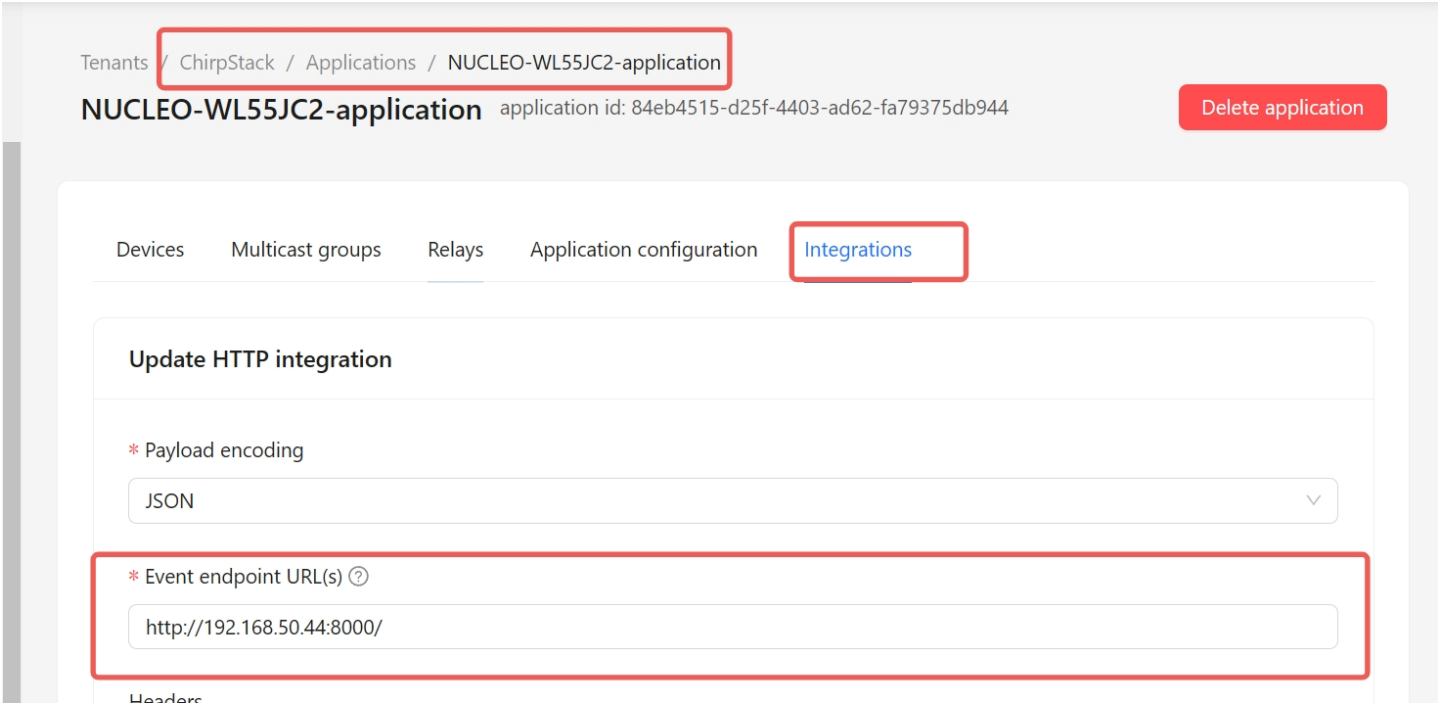
```
{
        "applicationID": "7",
        "applicationName": "test",        #应用服务器配置名称
        "deviceName": "WL55JC1",          #设备名称
        "devEUI": "AIDhFQAK2lE=",         #设备eui(base64编码)
        "rxInfo": [{
                "gatewayID": "qlVaAAAAAAA=",    #网关ID
                "time": null,
                "timeSinceGPSEpoch": null,
                "rssi": -72,
                "loRaSNR": 7.2,
                "channel": 6,
                "rfChain": 1,
                "board": 0,
                "antenna": 0,
                "location": {
                        "latitude": 0,
                        "longitude": 0,
                        "altitude": 0,
                        "source": "UNKNOWN",
                        "accuracy": 0
                },
                "fineTimestampType": "NONE",
                "context": "o6r2+A==",
                "uplinkID": "G9PiTDjWRlSwsPf1/+rWdw==",
                "crcStatus": "CRC_OK"
        }],
        "txInfo": {
                "frequency": 470700000,
                "modulation": "LORA",
                "loRaModulationInfo": {
                        "bandwidth": 125,
                        "spreadingFactor": 12,
                        "codeRate": "4/5",
                        "polarizationInversion": false
                }
        },
        "adr": true,
        "dr": 0,
        "fCnt": 1470,
        "fPort": 2,
```

```
        "data": "aGVsbG8hYXJleW91T0s/",
        "objectJSON": "",
        "tags": {},
        "confirmedUplink": false,
        "devAddr": "AdTOpg==",          #设备地址
        "publishedAt": "2020-09-29T15:02:14.642510060Z",
        "deviceProfileID": "ea3fb7fd-2afd-446a-89b6-ae2623b8c3c5",
        "deviceProfileName": "cn470_dev"
}
```

6. ChirpStack配置Integrations

6.1 http

配置http服务



6.2 mqtt

1. ChirpStack Application
   a. vim configuration/chirpstack/chirpstack.toml

```
[integration]
  enabled=["mqtt"]

  [integration.mqtt]
    server="tcp://host:post/"        #host/port (tcp/ssl具体要看目标mqtt服务配置)
    json=true
```

```

    username="test"                                #用户名
    password="test"                                #密码

    ca_cert="/etc/chirpstack/ca_certificates/ca.crt"            #证书
    tls_cert="/etc/chirpstack/ca_certificates/client.crt"
    tls_key="/etc/chirpstack/ca_certificates/client.key"
```

7. 测试

7.1 环境

a. 终端节点 NUCLEO-WL55JC2

   i. 配置方式 https://wiki.st.com/stm32mpu-ecosystem-v3/wiki/How_to_integrate_LoRaWAN_gateway#How_to_configure_Nucleo-WL55JC_as_LoRa_end_node_device

```
#define LORAWAN_DEVICE_EUI 00,80,E1,15,00,0A,DA,53
#define LORAWAN_JOIN_EUI 02,02,02,02,02,02,02,03
#define LORAWAN_DEVICE_ADDRESS                          01,FB,26,33
#define LORAWAN_APP_KEY 2B,7E,15,16,28,AE,D2,A6,AB,F7,15,88,09,CF,4F,33
#define LORAWAN_NWK_KEY 2B,7E,15,16,28,AE,D2,A6,AB,F7,15,88,09,CF,4F,33
#define LORAWAN_NWK_S_KEY 2B,7E,15,16,28,AE,D2,A6,AB,F7,15,88,09,CF,4F,33
#define LORAWAN_APP_S_KEY 2B,7E,15,16,28,AE,D2,A6,AB,F7,15,88,09,CF,4F,33
#define LORAMAC_SPECIFICATION_VERSION                   0x01000300
```

b. 网关RAK2287：https://store.rakwireless.com/products/wislink-concentrator-module-sx1302-rak2287-lorawan?variant=41826859385030

c. 开发板STM32MP1：https://wiki.st.com/stm32mpu-ecosystem-v3/wiki/How_to_integrate_LoRaWAN_gateway

d. ChirpStack：v4

```
git clone https://github.com/chirpstack/chirpstack-docker.git
cd chirpstack-docker
```

```

# 修改docker-compose.yml
# git diff
    volumes:
      - ./configuration/chirpstack-gateway-bridge:/etc/chirpstack-gateway-bridge
    environment:
-     - INTEGRATION__MQTT__EVENT_TOPIC_TEMPLATE=eu868/gateway/{{ .GatewayID }}/event/{{ .EventType }}
-     - INTEGRATION__MQTT__STATE_TOPIC_TEMPLATE=eu868/gateway/{{ .GatewayID }}/state/{{ .StateType }}
-     - INTEGRATION__MQTT__COMMAND_TOPIC_TEMPLATE=eu868/gateway/{{ .GatewayID }}/command/#
+     - INTEGRATION__MQTT__EVENT_TOPIC_TEMPLATE=cn470_10/gateway/{{ .GatewayID }}/event/{{ .EventType }}
+     - INTEGRATION__MQTT__STATE_TOPIC_TEMPLATE=cn470_10/gateway/{{ .GatewayID }}/state/{{ .StateType }}
+     - INTEGRATION__MQTT__COMMAND_TOPIC_TEMPLATE=cn470_10/gateway/{{ .GatewayID }}/command/#
    depends_on:
      - mosquitto

  chirpstack-gateway-bridge-basicstation:
    image: chirpstack/chirpstack-gateway-bridge:4
    restart: unless-stopped
-   command: -c /etc/chirpstack-gateway-bridge/chirpstack-gateway-bridge-basicstation-eu868.toml
+   command: -c /etc/chirpstack-gateway-bridge/chirpstack-gateway-bridge-basicstation-cn470_10.toml
    ports:
      - 3001:3001
    volumes:

# 启动
sudo docker-compose up
```

7.2 部署

1. 开发板STM32MP1：192.168.50.130
   a. 按照https://wiki.st.com/stm32mpu-ecosystem-v3/wiki/How_to_integrate_LoRaWAN_gateway此文档刷机成功后,配置网关
   b. 启动网关驱动程序

      直接通过udp将数据转发到远程chirpstack服务(chirpstack-gateway-bridge)

```
cd /usr/local/lorawan-gateway/rak2287
# 修改 gateway_conf
    "gateway_ID": "24cd8dffff54007d",        # 网关ID
    "server_address": "192.168.50.44",       # chirpstack 服务host
    "serv_port_up": 1700,                    # chirpstack 服务port
    "serv_port_down": 1700,

# 启动
./lora_pkt_fwd
```

a. 如何编译调试lora_pkt_fwd

0001-stm32mp1-rak2287.patch

layers/meta-st/meta-st-stm32mpu-app-lorawan/recipes-framework/rak-2287/rak-2287/0001-stm32mp1-rak2287.patch

```
git clone https://github.com/Lora-net/sx1302_hal.git
cd sx1302_hal/ && git apply 0001-stm32mp1-rak2287.patch
make -j4
```

2. clond:192.168.50.44

a. http

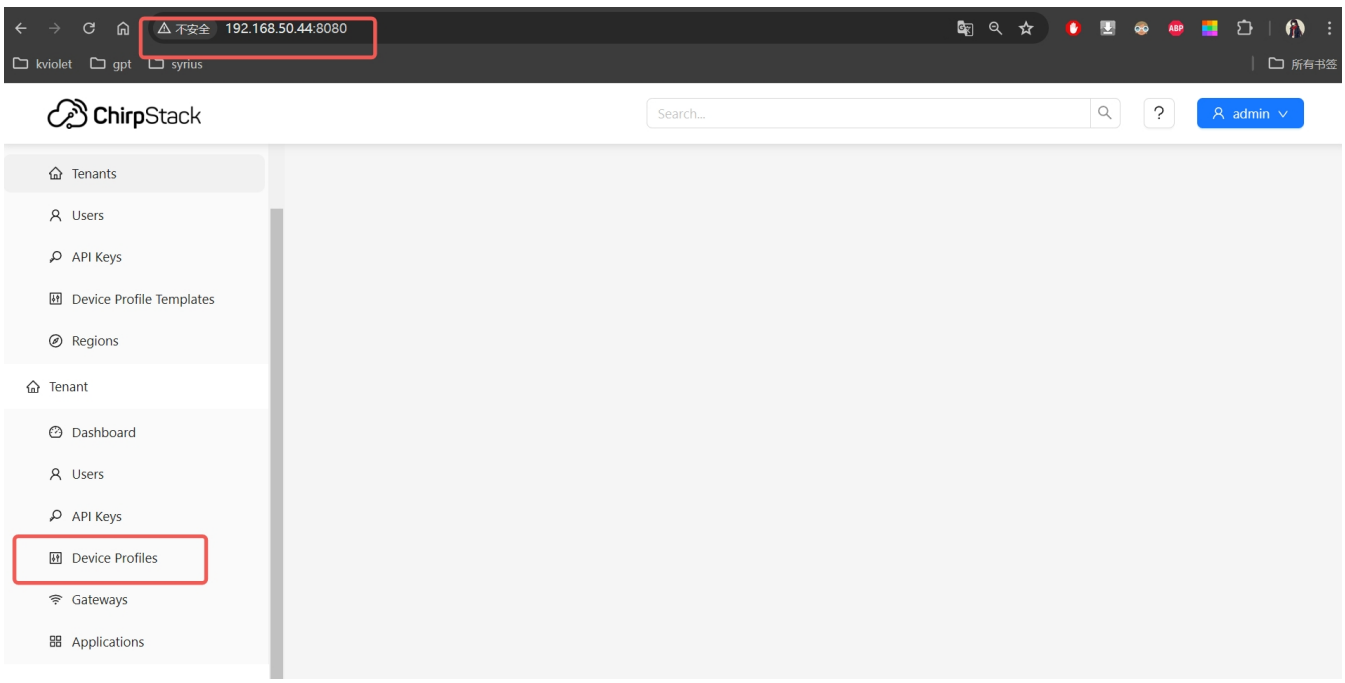
```
# simple_web_server.py
from http.server import BaseHTTPRequestHandler, HTTPServer

# 定义一个自定义请求处理器类
class SimpleHTTPRequestHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        # 发送响应状态码
        self.send_response(200)
        # 发送响应头
        self.send_header('Content-type', 'text/html')
        self.end_headers()
        # 发送响应体
        message = b"Hello, World!"
        self.wfile.write(message)

    def do_POST(self):
        # 获取请求内容长度
        content_length = int(self.headers['Content-Length'])
        # 读取请求内容
```

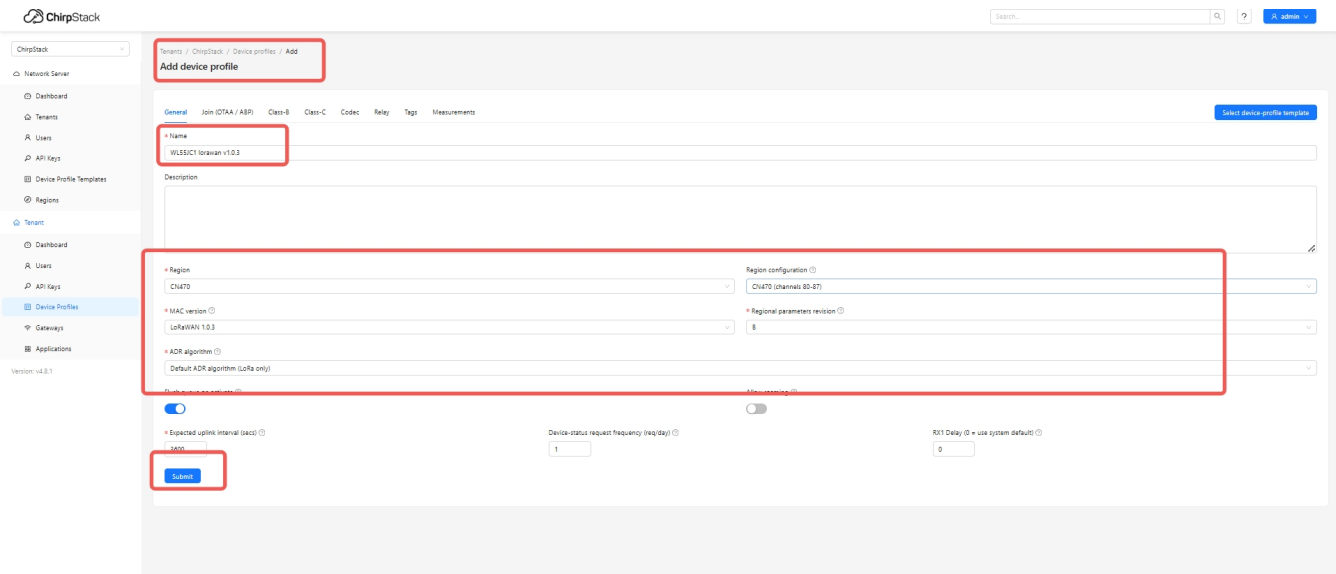
```python
        post_data = self.rfile.read(content_length)
        # 发送响应状态码
        self.send_response(200)
        # 发送响应头
        self.send_header('Content-type', 'text/html')
        self.end_headers()
        # 发送响应体
        print(post_data)
        message = b""
        self.wfile.write(message)

# 设置服务器地址和端口
server_address = ('0.0.0.0', 8000)
# 创建HTTPServer对象
httpd = HTTPServer(server_address, SimpleHTTPRequestHandler)

# 启动服务器
print(f"Starting HTTP server on {server_address}:{server_address[1]}")
httpd.serve_forever()

```

b. mqtt

3. chirpstack：192.168.50.44

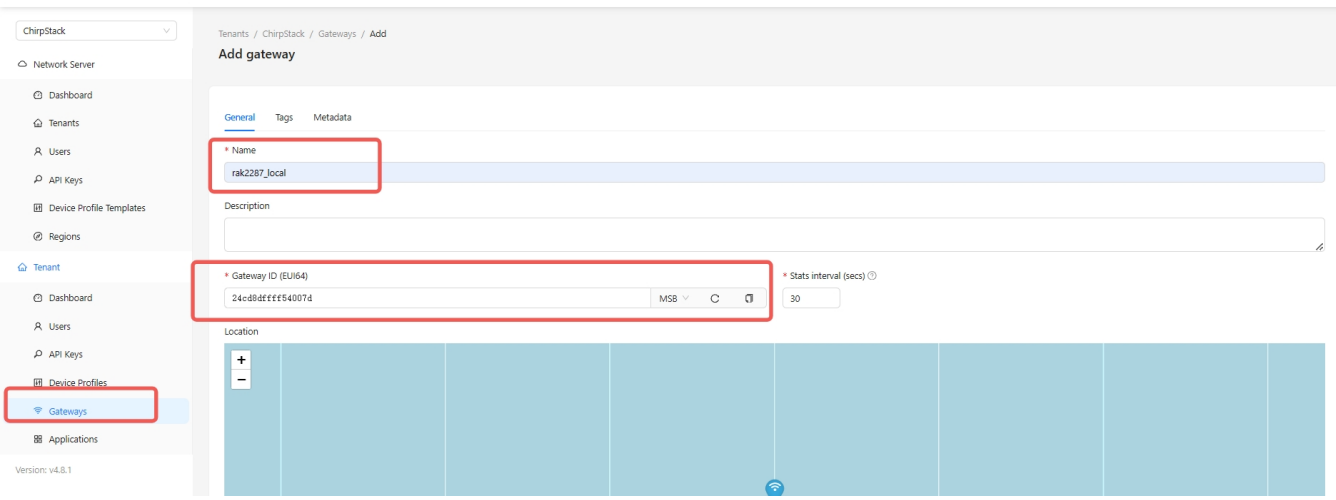
a. 配置Device profiles



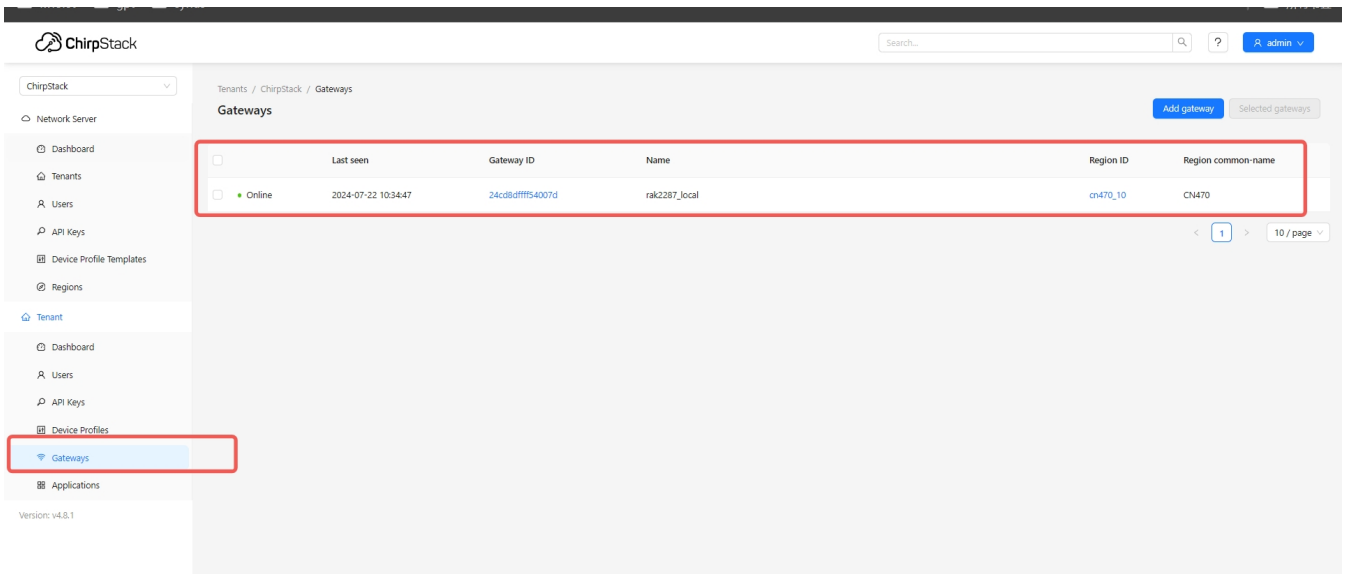


b. 配置Gateways

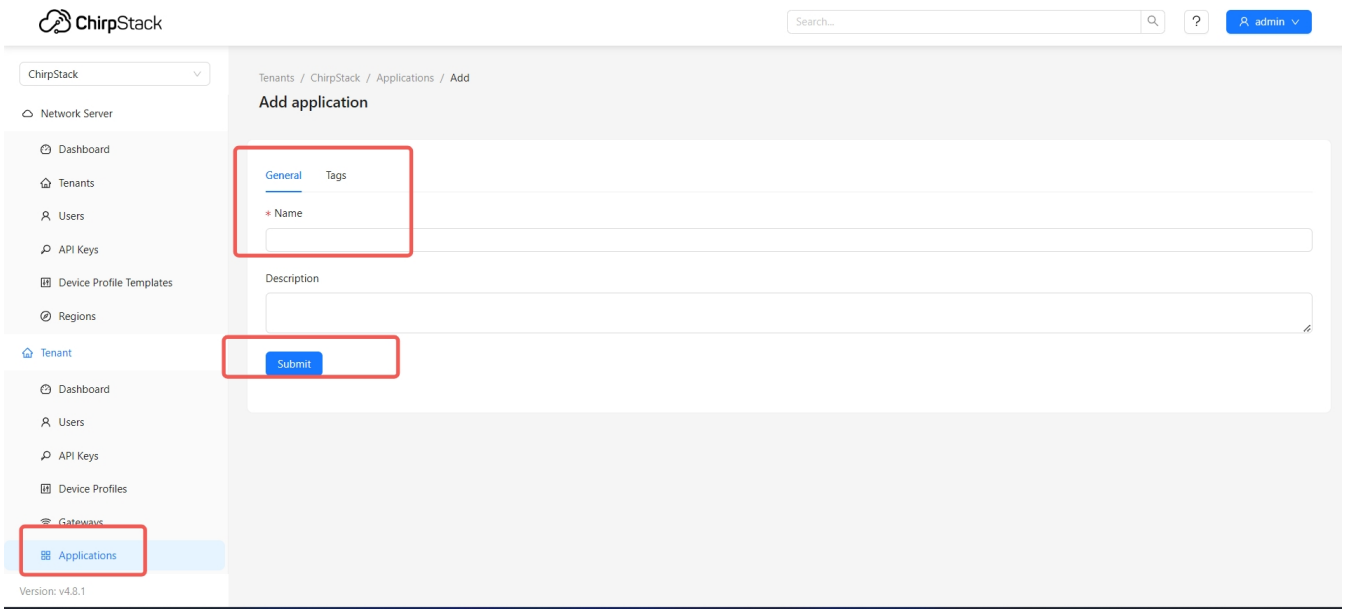
i. 配置



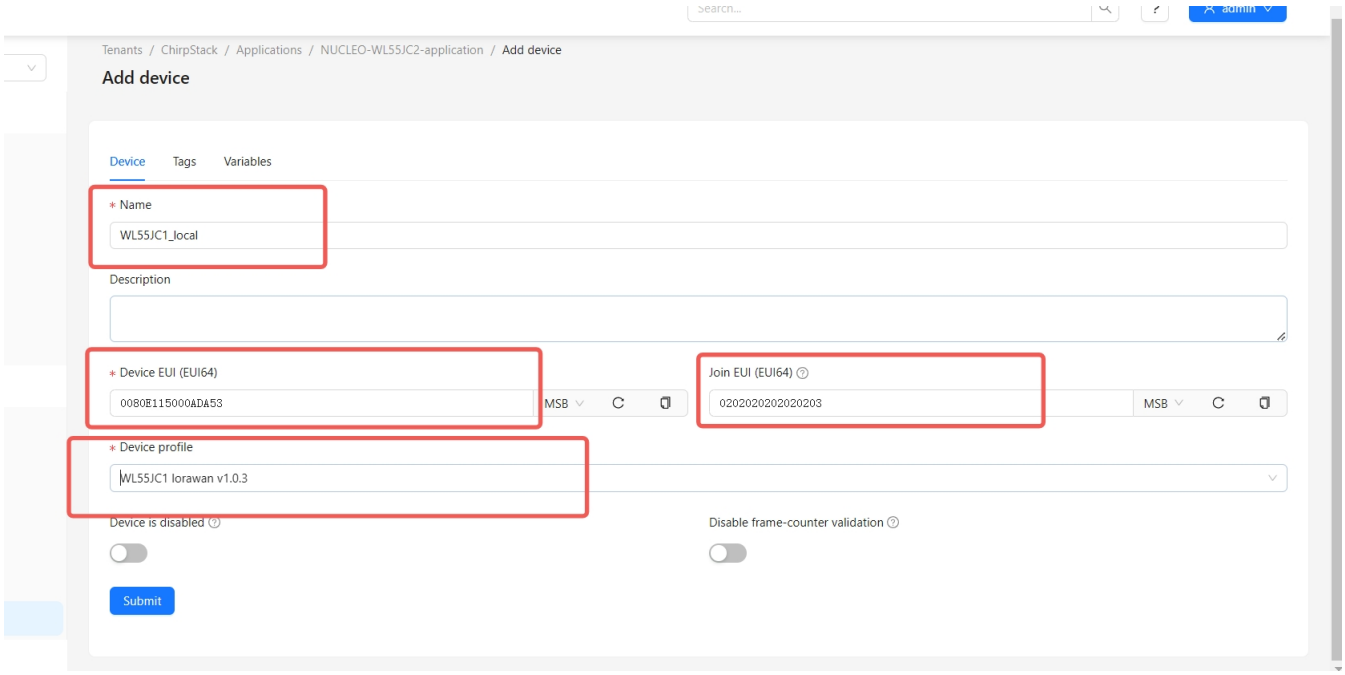
i. 网关注册成功(需要网关驱动lora_pkt_fwd转发成功)



c. 配置Applications



key：LoRaWAN网络



7.3 连接成功日志

a. NUCLEO-WL55JC2串口日志

```
[10:46:00:865] APPLICATION_VERSION: V1.3.0
[10:46:00:876] MW_LORAWAN_VERSION:  V2.5.0
[10:46:00:876] MW_RADIO_VERSION:   V1.3.0
[10:46:00:876] L2_SPEC_VERSION:    V1.0.3
[10:46:00:876] RP_SPEC_VERSION:    V1-1.0.3
[10:46:00:889] ###### AppKey:     2B:7E:15:16:28:AE:D2:A6:AB:F7:15:88:09:CF:4F:33
[10:46:00:895] ###### NwkKey:     2B:7E:15:16:28:AE:D2:A6:AB:F7:15:88:09:CF:4F:33
[10:46:00:902] ###### AppSKey:    2B:7E:15:16:28:AE:D2:A6:AB:F7:15:88:09:CF:4F:33
[10:46:00:909] ###### NwkSKey:    2B:7E:15:16:28:AE:D2:A6:AB:F7:15:88:09:CF:4F:33
[10:46:00:914] ###### DevEUI:     00:80:E1:15:00:0A:DA:53
[10:46:00:920] ###### AppEUI:     02:02:02:02:02:02:02:03
[10:46:00:925] ###### DevAddr:    01:FB:26:33
[10:46:00:925] 0s036:TX on freq 476300000 Hz at DR 0
```

```
[10:46:19:251] ###### = JOINED = OTAA ====================
[10:46:19:251] ###### MCRootKey:  5A:7A:7F:50:16:AC:A9:E6:78:8A:05:59:A1:8A:CE:F2
[10:46:19:257] ###### MCKEKey:    57:3D:35:CE:B4:4E:08:25:9D:30:E8:A4:A9:D2:97:C2
[10:46:19:257] ###### AppSKey:    5B:5F:56:3D:FD:DD:30:2D:31:85:94:70:DF:BD:DD:F3
[10:46:19:264] ###### NwkSKey:    0B:77:6C:B4:D1:8E:B0:67:78:BE:9A:83:C9:C3:81:F6
[10:46:19:270] ###### DBIntKey:   FE:D7:71:30:F8:9F:86:25:DF:23:24:8B:31:6F:20:FB
[10:46:19:275] ###### DevEUI:     00:80:E1:15:00:0A:DA:53
[10:46:19:281] ###### AppEUI:     02:02:02:02:02:02:02:03
[10:46:19:286] ###### DevAddr:    00:9D:9F:AC
[10:46:20:915] 20s047:VDDA: 254
[10:46:20:932] 20s047:temp: 28
[10:46:20:932] 20s051:TX on freq 487300000 Hz at DR 0
[10:46:20:932] 20s054:SEND REQUEST
[10:46:22:568] 21s701:MAC txDone
[10:46:23:600] 22s732:RX_1 on freq 507700000 Hz at DR 0
[10:46:23:798] 22s930:IRQ_RX_TX_TIMEOUT
[10:46:23:800] 22s930:MAC rxTimeOut
[10:46:24:600] 23s732:RX_2 on freq 505300000 Hz at DR 0
[10:46:24:798] 23s930:IRQ_RX_TX_TIMEOUT
[10:46:24:800] 23s930:MAC rxTimeOut
[10:46:24:815]
```

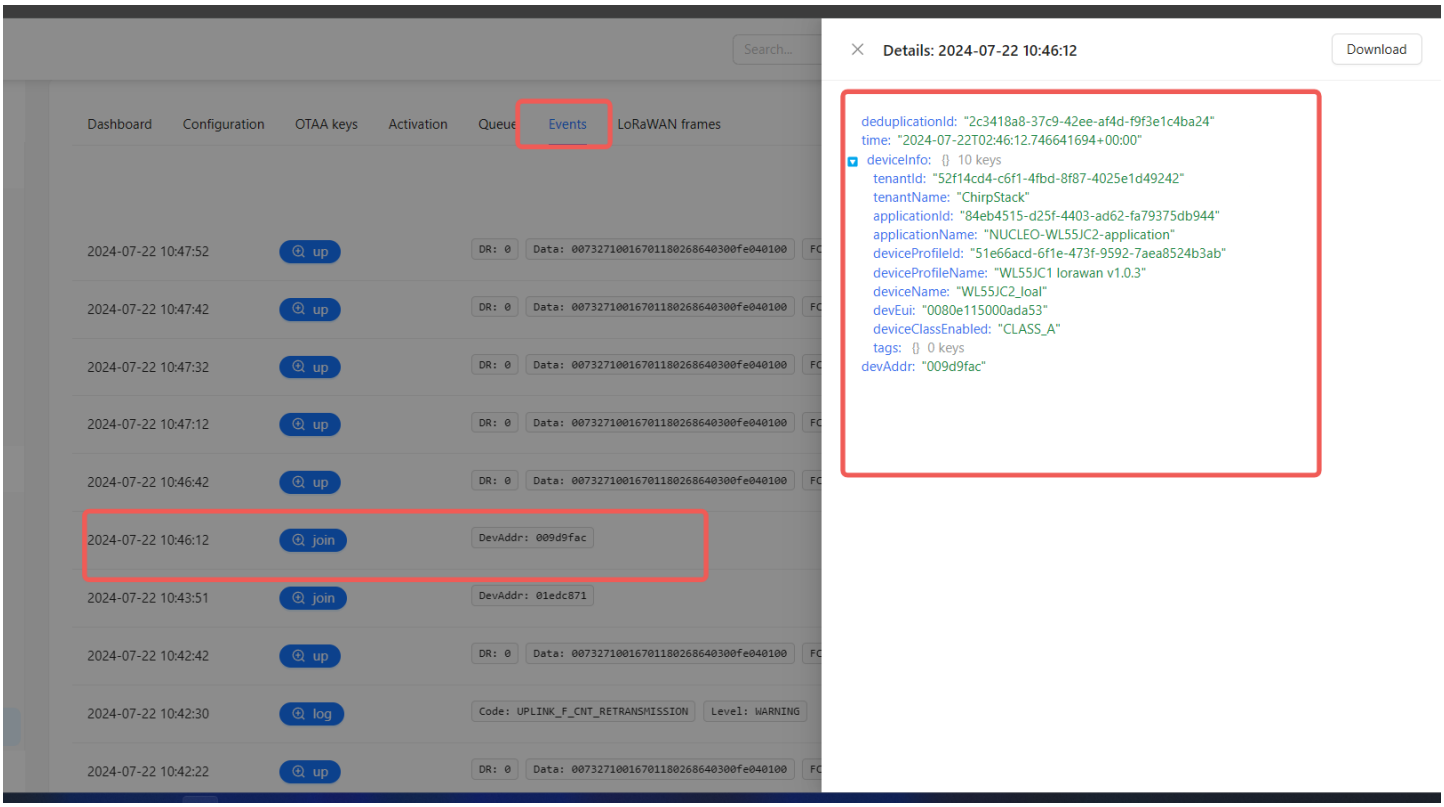
b. 网关驱动程序日志

```
INFO: Received pkt from mote: 02020203 (fcnt=514)

JSON up: {"rxpk":[{"jver":1,"tmst":13476843,"chan":4,"rfch":1,"freq":470.300000,"mid": 0,"stat":1,"modu":"LORA","datr":"SF12BW125","c
odr":"4/5","rssis":-24,"lsnr":10.2,"foff":24,"rssi":-23,"size":23,"data":"AAMCAgICAgICU9oKABXhgADRLNFTMgk="}]}
INFO: [up] PUSH_ACK received in 23 ms
INFO: [down] PULL_RESP received  - token[1:163] :)

JSON down: {"txpk":{"imme":false,"rfch":0,"powe":19,"ant":0,"brd":0,"tmst":18476843,"freq":500.3,"modu":"LORA","datr":"SF12BW125","co
dr":"4/5","ipol":true,"size":33,"data":"IM7K+PXMq1kdkXspGIolAlgQhjHeXBl3Xvq4k07dpLz2"}}
lora_packet_time_on_air:149: INFO: LoRa packet ToA: 1810432 us (n_symbol:55.250000, t_symbol_us:32768)
lgw_time_on_air:1637: INFO: LoRa packet ToA: 1810 ms
Note: SPI write success
Note: SPI write success
```

c. chirpstack 日志

```

chirpstack_1                                | 2024-07-22T02:46:12.638385Z  INFO up{deduplication_id=2c3418a8-37c9-42ee-af4d-f9f3e1c4ba24}: chirpstack::uplink: Uplink received m_type=JoinRequest
chirpstack_1                                | 2024-07-22T02:46:12.731762Z  INFO up{deduplication_id=2c3418a8-37c9-42ee-af4d-f9f3e1c4ba24}:join_request{dev_eui="0080e115000ada53"}: chirpstack::storage::device_keys: Device-nonce validated, join-nonce incremented and stored dev_eui=0080e115000ada53 dev_nonce=11473
chirpstack_1                                | 2024-07-22T02:46:12.737449Z  INFO up{deduplication_id=2c3418a8-37c9-42ee-af4d-f9f3e1c4ba24}:join_request{dev_eui="0080e115000ada53"}: chirpstack::storage::device_queue: Device queue flushed dev_eui=0080e115000ada53 count=0
```

```
chirpstack_1                                | 2024-07-22T02:46:12.745083Z  INFO up{deduplication_id=2c3418a8-37c9-42ee-af4d-f9f3e1c4ba24}:join_request{dev_eui="0080e115000ada53"}: chirpstack::storage::device: Device partially updated dev_eui=0080e115000ada53
chirpstack_1                                | 2024-07-22T02:46:12.746321Z  INFO up{deduplication_id=2c3418a8-37c9-42ee-af4d-f9f3e1c4ba24}:join_request{dev_eui="0080e115000ada53"}:join_accept{downlink_id=447259393}: chirpstack::storage::downlink_frame: Downlink-frame saved downlink_id=447259393
chirpstack_1                                | 2024-07-22T02:46:12.746506Z  INFO up{deduplication_id=2c3418a8-37c9-42ee-af4d-f9f3e1c4ba24}:join_request{dev_eui="0080e115000ada53"}:join_accept{downlink_id=447259393}: chirpstack::gateway::backend::mqtt: Sending downlink frame region_id=cn470_10 gateway_id=24cd8dffff54007d topic=cn470_10/gateway/24cd8dffff54007d/command/down json=false
chirpstack-gateway-bridge_1                 | time="2024-07-22T02:46:12.748171448Z" level=info msg="integration/mqtt: downlink frame received" downlink_id=447259393 gateway_id=24cd8dffff54007d
chirpstack_1                                | 2024-07-22T02:46:12.759110Z  INFO chirpstack::integration::http: Posting event event=join url=http://192.168.50.44:8000/
chirpstack_1                                | 2024-07-22T02:46:12.759883Z  INFO chirpstack::integration::mqtt: Publishing event topic=application/84eb4515-d25f-4403-ad62-fa79375db944/device/0080e115000ada53/event/join
```



7.4 数据上传成功日志

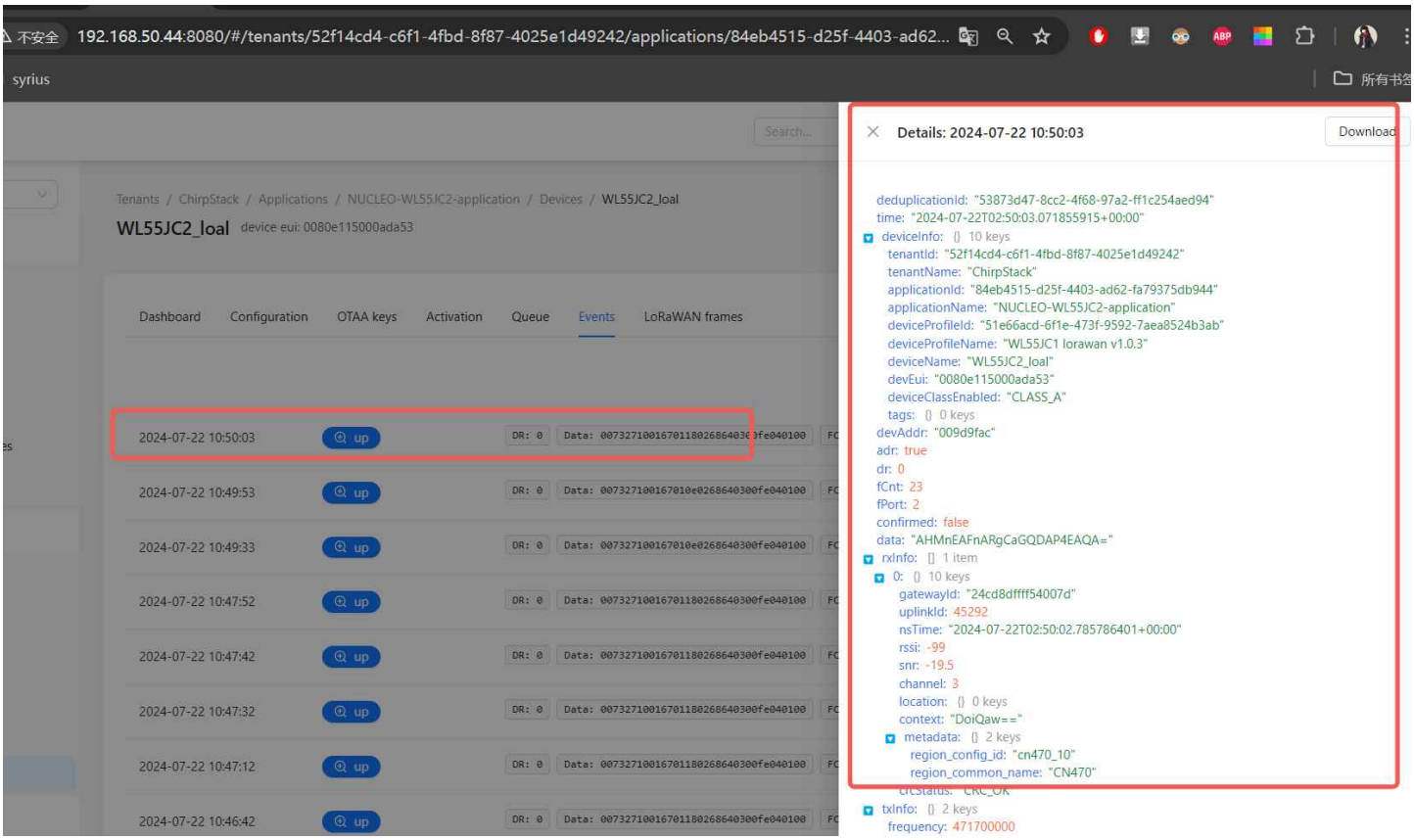
1. NUCLEO-WL55JC2串口日志

```
[10:49:34:978] ###### ========== MCPS-Confirm ==============
[10:49:41:088] 220s204:VDDA: 254
[10:49:41:103] 220s204:temp: 28
[10:49:41:103] 220s208:TX on freq 486300000 Hz at DR 0
[10:49:41:103] 220s210:SEND REQUEST
[10:49:42:741] 221s857:MAC txDone
[10:49:43:773] 222s888:RX_1 on freq 506700000 Hz at DR 0
[10:49:43:971] 223s086:IRQ_RX_TX_TIMEOUT
[10:49:43:986] 223s086:MAC rxTimeOut
[10:49:44:773] 223s888:RX_2 on freq 505300000 Hz at DR 0
[10:49:44:971] 224s086:IRQ_RX_TX_TIMEOUT
[10:49:44:974] 224s086:MAC rxTimeOut
[10:49:44:988]
[10:49:44:988] ###### ========== MCPS-Confirm ==============
[10:49:51:096] 230s211:VDDA: 254
[10:49:51:113] 230s211:temp: 27
[10:49:51:113] 230s215:TX on freq 486700000 Hz at DR 0
[10:49:51:113] 230s218:SEND REQUEST
[10:49:52:750] 231s865:MAC txDone
[10:49:53:782] 232s896:RX_1 on freq 507100000 Hz at DR 0
[10:49:54:450] 233s565:IRQ_RX_TX_TIMEOUT
[10:49:54:454] 233s565:MAC rxTimeOut
[10:49:54:781] 233s896:RX_2 on freq 505300000 Hz at DR 0
[10:49:54:979] 234s094:IRQ_RX_TX_TIMEOUT
[10:49:54:982] 234s094:MAC rxTimeOut
[10:49:54:998]
[10:49:54:998] ###### ========== MCPS-Confirm ==============
```

2. 网关驱动程序日志

```
INFO: Received pkt from mote: 009D9FAC (fcnt=23)

JSON up: {"rxpk":[{"jver":1,"tmst":243830891,"chan":3,"rfch":0,"freq":471.700000,"mid": 0,"stat":1,"modu":"LORA","datr":"SF12BW125","c
odr":"4/5","rssis":-118,"lsnr":-19.5,"foff":30,"rssi":-99,"size":30,"data":"QKyfnQCAFwACPinRIKIC3HXN12Rydi8Le9dPKYhp"}]}
INFO: [up] PUSH_ACK received in 8 ms
INFO: [down] PULL_RESP received  - token[13:111] :)

JSON down: {"txpk":{"imme":false,"rfch":0,"powe":19,"ant":0,"brd":0,"tmst":244830891,"freq":501.7,"modu":"LORA","datr":"SF12BW125","c
odr":"4/5","ipol":true,"size":17,"data":"YKyfnQCFBwARAAAAAmMWLKI="}}
lora_packet_time_on_air:149: INFO: LoRa packet ToA: 1318912 us (n_symbol:40.250000, t_symbol_us:32768)
lgw_time_on_air:1637: INFO: LoRa packet ToA: 1319 ms
```

3. chirpstack 日志



4. clone日志

```
172.19.0.6 - - [22/Jul/2024 10:51:53] "POST /?event=up HTTP/1.1" 200 -
b'{"deduplicationId":"2e6ad30e-7795-48f9-a10d-49d005bf6622","time":"2024-07-22T02:51:53.165073018+00:00","deviceInfo":{"tenantId":"52f14cd4-c
6f1-4fbd-8f87-4025e1d49242","tenantName":"ChirpStack","applicationId":"84eb4515-d25f-4403-ad62-fa79375db944","applicationName":"NUCLEO-WL55JC
2-application","deviceProfileId":"51e66acd-6f1e-473f-9592-7aea8524b3ab","deviceProfileName":"WL55JC1 lorawan v1.0.3","deviceName":"WL55JC2_lo
al","devEui":"0080e115000ada53","deviceClassEnabled":"CLASS_A","tags":{}},"devAddr":"009d9fac","adr":true,"dr":0,"fCnt":34,"fPort":2,"confirm
ed":false,"data":"AHMnEAFnARgCaGQDAP4EAQA=","rxInfo":[{"gatewayId":"24cd8dffff54007d","uplinkId":63881,"nsTime":"2024-07-22T02:51:52.88037932
1+00:00","rssi":-100,"snr":-20.0,"channel":1,"location":{},"context":"FRhr5A==","metadata":{"region_common_name":"CN470","region_config_id":"
cn470_10"},"crcStatus":"CRC_OK"}],"txInfo":{"frequency":471300000,"modulation":{"lora":{"bandwidth":125000,"spreadingFactor":12,"codeRate":"C
R_4_5"}}}}'
```

7.5 配置MQTT证书

生成证书：MQTT

7.5.1 网关->网络服务器

a. 网关测配置

i. UDP Packet Forwarder LoRaWAN网络 直接通过udp协议转发数据，需要提供额外的加解密算法对数据进行加密并在接收端解密

ii. UDP Packet Forwarder + ChirpStack MQTT Forwarder LoRaWAN网络

1. 配置

```
# 下载 https://www.chirpstack.io/docs/chirpstack-mqtt-forwarder/downloads.html
```

```
# 配置 chirpstack-mqtt-forwarder.toml
# 需要注意 chirpstack-mqtt-forwarder 订阅/发布topic必须与 chirpstack-gateway-bridge一致

  topic_prefix="cn470_10"
  server="ssl://192.168.50.44:1884" # 跨平台通讯 目标mqtt服务服务需要证书
  username=""
  password=""
  ca_cert=""
  tls_cert=""
  tls_key=""


 #驱动程序 lora_pkt_fwd 配置信息
    "gateway_ID": "24cd8dffff54007d",
    "server_address": "localhost",
    "serv_port_up": 1700,
    "serv_port_down": 1700,
```

2. 启动

```
./chirpstack-mqtt-forwarder -c chirpstack-mqtt-forwarder.toml
```

b. 网络服务器配置

i. vim configuration/mosquitto/config/mosquitto.conf

```
listener 1883                #chirpstack内部通讯端口
allow_anonymous true


listener 1884                #ChirpStack MQTT Forwarder端口
protocol mqtt
cafile /mosquitto/config/ca_certificates/ca.crt
certfile /mosquitto/config/ca_certificates/syrius.crt
keyfile /mosquitto/config/ca_certificates/syrius.key
require_certificate true
```

7.5.2 应用服务器->clond

i. clond mosquitto配置

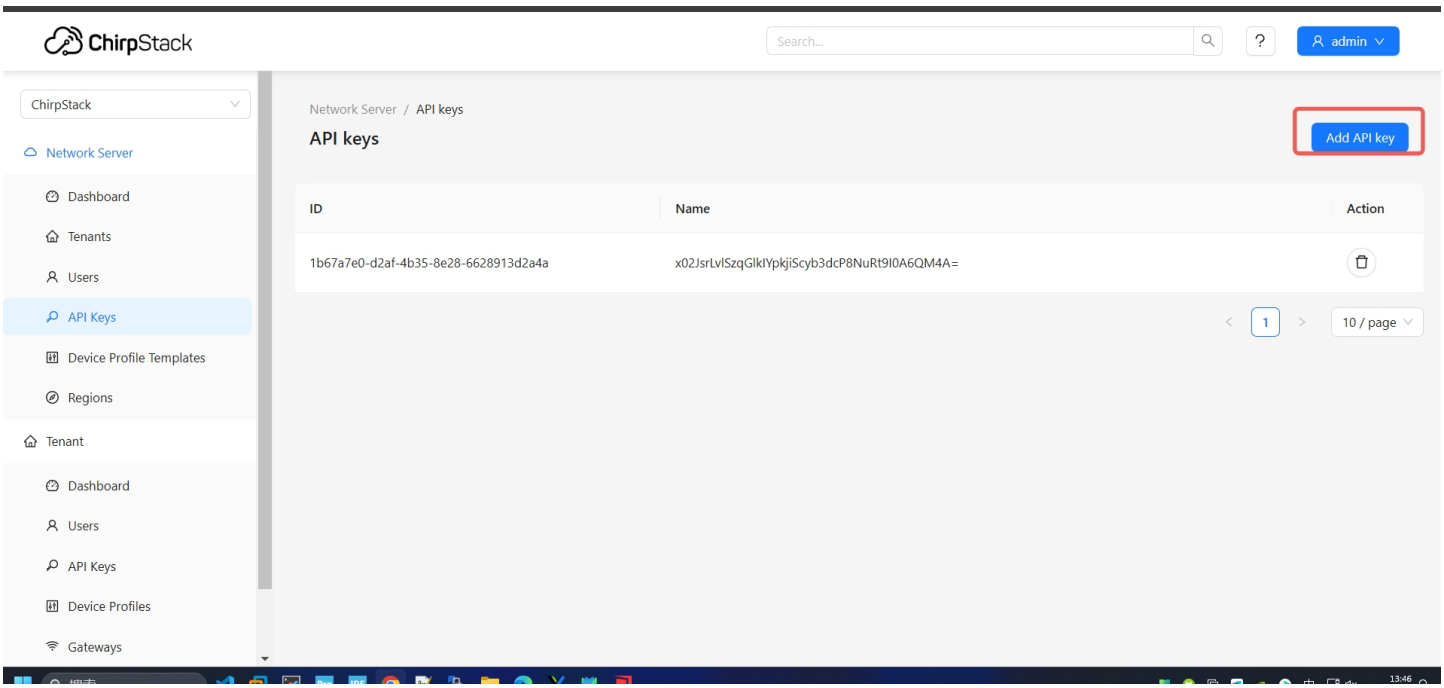
1. vim mosquitto.conf

```
# Place your local configuration in /etc/mosquitto/conf.d/
#
# A full description of the configuration file is at
# /usr/share/doc/mosquitto/examples/mosquitto.conf.example

listener 1883 0.0.0.0                        # port hose
allow_anonymous false                        # 取消匿名登录
password_file /etc/mosquitto/pwfile          # 设置账号密码

cafile /etc/mosquitto/ca_certificates/ca.crt            # 证书
certfile /etc/mosquitto/ca_certificates/syrius.crt      # 证书
keyfile /etc/mosquitto/ca_certificates/syrius.key       # key
require_certificate true                                # 双向验证

```

ii. 应用服务器配置：LoRaWAN网络

7.6 API

1. 登录ChirpStack web界面生成api key



2. 云端gRPC请求
   a. https://github.com/chirpstack/chirpstack 下载proto文件
   b. 验证

Protos

Filter methods

- common.proto
- application.proto
- device.proto
  - api.DeviceService
    - Create
    - Get
    - Update
    - Delete
    - List
    - CreateKeys
    - GetKeys
    - UpdateKeys
    - DeleteKeys
    - FlushDevNonces
    - Activate
    - Deactivate
    - GetActivation
    - GetRandomDevAddr
    - GetMetrics

api.DeviceService.GetKeys

Env | 192.168.50.44:8080 | Unary Call | TLS | GRPC | Manual | View Proto

Editor

```
1 {
2   "dev_eui": "0080e115000ada53"
3 }
```

请求

Response 响应

0.079s

```
{
  "device_keys": {
    "dev_eui": "0080e115000ada53",
    "nwk_key": "2b7e151628aed2a6abf7158809cf4f33",
    "app_key": "00000000000000000000000000000000"
  },
  "created_at": {
    "seconds": "1721354637",
    "nanos": 383886000
  },
  "updated_at": {
    "seconds": "1721616372",
    "nanos": 664420000
  }
}
```

METADATA

```
1 {"authorization":"Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJhdWQiOiJjaGlycHN0YWNrIiwiZXhwIjoxNzIxNzk5ODI3LCJpc3MiOiJjaGlycHN0YWNrIiwic3ViIjoiMDUyM
```

设置token

搜索 英 13:52 2024/7/23